perforce OpenLogic

BLOG October 28, 2025

Using Kafka With ZooKeeper

Joe Carder

APACHE KAFKA, MIDDLEWARE

Apache Kafka and ZooKeeper have long been two of the most successful open source products in the computing world. From small startups to Fortune 100 enterprises, Kafka — once tightly coupled with ZooKeeper — has powered modern data streaming architectures for over a decade.

However, Kafka's architecture has evolved significantly. Starting in Apache Kafka 3.3 and finalized in Kafka 4.0 (released in 2024), Kafka no longer requires ZooKeeper at all. The modern KRaft (Kafka Raft) mode fully replaces ZooKeeper for metadata management, controller elections, and cluster coordination.

This blog explains how Kafka and ZooKeeper originally worked together, and how that model has been replaced by the new KRaft architecture that now underpins all production Kafka clusters.

Table of Contents

1. What Is Apache Kafka?
2. Apache Kafka and ZooKeeper Overview
3. Kafka Brokers and Topics
4. What Is Apache ZooKeeper?
5. How Kafka and ZooKeeper Worked Together (Legacy Architecture)
6. The Shift From ZooKeeper to KRaft Mode
7. Is ZooKeeper Still Useful?
8. Final Thoughts

Table of Contents

Back to top

What Is Apache Kafka?

> *Apache Kafka is a distributed event streaming platform that moves data from producers to consumers in real time. Producers write data to Kafka topics, and consumers read from those topics. Topics are divided into partitions, allowing messages to be distributed across multiple brokers for scalability and fault tolerance. Kafka's append-only log and replication model enable it to achieve extremely high throughput — ranging from thousands to millions of messages per second.*

Back to top

Apache Kafka and ZooKeeper Overview

Kafka was originally developed by LinkedIn and open sourced to the Apache Software Foundation in 2011, graduating from the Apache Incubator on October 23, 2012. It was named after the author Franz Kafka by its lead developer, Jay Kreps, because it is a "system optimized for writing."

ZooKeeper, in contrast, was developed by Yahoo! in the early 2000s as a Hadoop sub-project, designed to simplify coordination in distributed systems. It provides services such as configuration management, leader election, synchronization, and naming through a hierarchical key-value store.

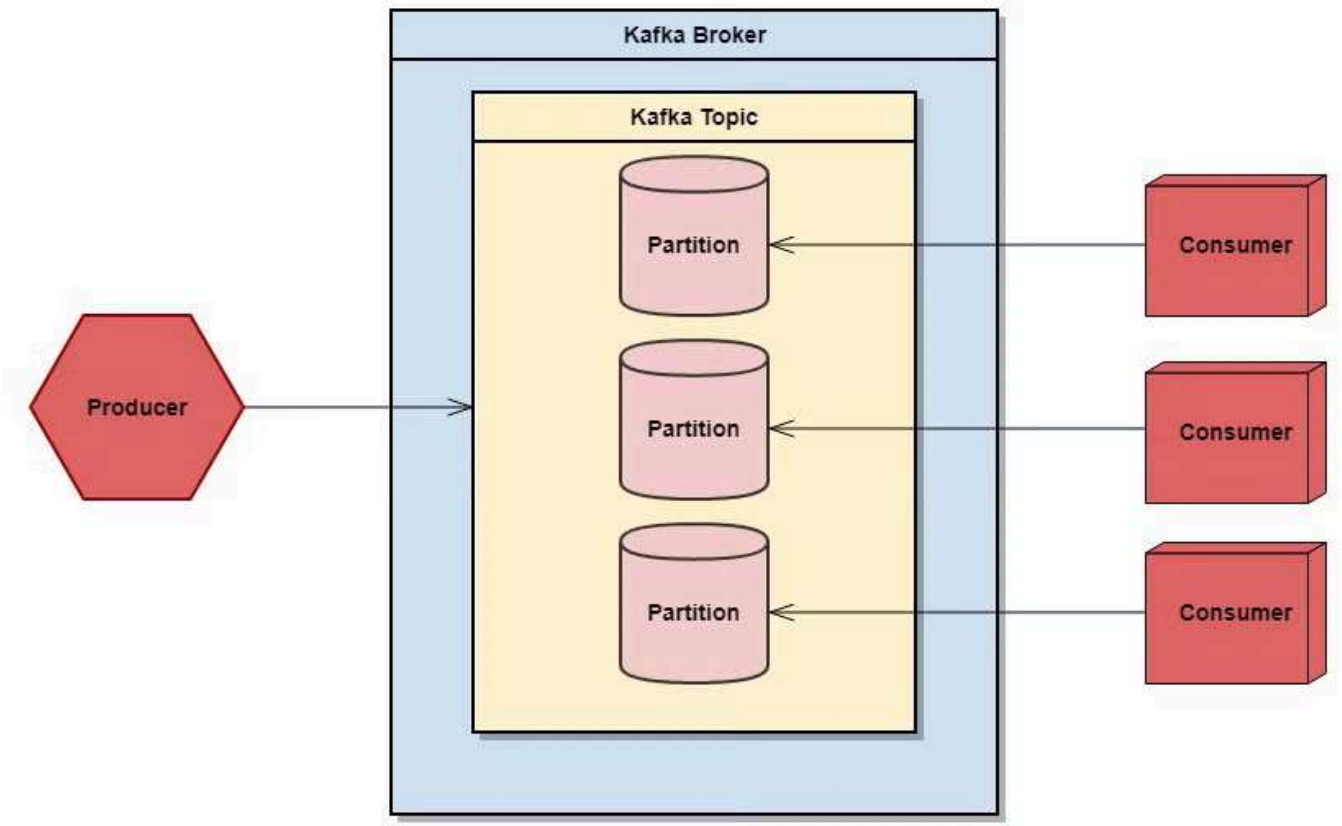
Kafka originally relied on ZooKeeper to manage cluster metadata, broker registration, and controller elections—but that dependency has now been fully removed.

Back to top

Kafka Brokers and Topics

A Kafka broker handles client requests (from both producers and consumers) and manages the replication of topic partitions. Each broker is part of a Kafka cluster that collectively stores and serves topic data.

A Kafka topic is a category or feed name to which records are published. Each topic consists of one or more partitions, and each partition is an ordered, immutable sequence of records identified by an offset. This partitioned design enables horizontal scalability and parallel consumption.



Back to top

What Is Apache ZooKeeper?

> *ZooKeeper provides a highly reliable control plane for distributed coordination of clustered applications through a hierarchical key-value store. The suite of services provided by ZooKeeper include distributed configuration services, synchronization services, leadership election services, and a naming registry.*

A core concept of ZooKeeper is the znode. A ZooKeeper znode is a data node that is tracked by a stat structure including data changes, ACL changes, a version number, and timestamp. The version number, along with the timestamp, allows ZooKeeper to coordinate updates and cache information. Each change to a znode's data results in a version change which is used to makes sure znode changes are applied correctly.

The Decision Maker's Guide to Apache Kafka

A comprehensive guide with tips for implementing Kafka, security best practices, partitioning strategies, use cases, and more.

Get My Copy ▶

Back to top

How Kafka and ZooKeeper Worked Together (Legacy Architecture)

In pre–KRaft Kafka clusters (Kafka 2.x through early 3.x releases), ZooKeeper was a critical dependency. ZooKeeper handled:

- Broker registration and cluster membership
- Controller election (determining which broker managed partition leadership)
- Topic metadata and configuration management
- Detection of broker failures
- Access control list (ACL) storage

Kafka brokers would connect to ZooKeeper to register themselves and watch for changes in cluster state. If a broker failed, ZooKeeper would trigger a controller election to assign new leaders for affected partitions.

This model worked well for many years, but as Kafka scaled into clusters with thousands of brokers and topics, ZooKeeper coordination became a performance bottleneck and operational complexity grew.

Back to top

The Shift From ZooKeeper to KRaft Mode

Introduced by KIP-500 and first released experimentally in Kafka 2.8.0 (2021), KRaft mode (Kafka Raft Metadata mode) removes ZooKeeper entirely. Kafka now uses an internal Raft-based quorum controller to manage metadata and perform leader elections natively within brokers.

Since Kafka 3.3, KRaft mode has been considered production-ready. As of Kafka 4.0, ZooKeeper-based clusters are deprecated and no longer supported. All new Kafka clusters now run in KRaft mode by default.

How KRaft Works

In KRaft mode, metadata management is handled by a controller quorum — a small set of dedicated or co-located brokers that replicate cluster metadata using the Raft consensus protocol.

- Leader election: One controller is elected leader via Raft consensus.
- Followers: Other controllers replicate the leader's metadata log and can take over if the leader fails.
- Metadata log: All cluster metadata (topics, partitions, configurations, ACLs, etc.) is written to a replicated log managed by the quorum.
- Brokers: Regular brokers subscribe to updates from the controller quorum.

This approach reduces external dependencies, simplifies operations, and improves scalability and recovery performance, especially during rebalancing or broker restarts.

Deprecated: ZooKeeper-Specific Features

With the move to KRaft, ZooKeeper-based mechanisms have been retired:

- Controller election via /controller ephemeral node → replaced by Raft-based quorum election
- ZooKeeper znode structure → replaced by an internal metadata log
- ACLs stored in ZooKeeper → now managed directly in Kafka metadata
- Broker registration through ZooKeeper ephemeral znodes → replaced by internal broker heartbeats

If you are still running a ZooKeeper-based cluster, Kafka migration tools are available (starting in Kafka 3.5) to convert cluster metadata from ZooKeeper to KRaft mode. Full ZooKeeper removal is complete as of Kafka 4.0.

Back to top

Is ZooKeeper Still Useful?

For Kafka — no. Kafka's ZooKeeper dependency is fully deprecated.

However, ZooKeeper itself remains an active Apache project and continues to be used by other open source projects such as Hadoop, HBase, Solr, Spark, and NiFi.

Organizations running older Kafka clusters (pre-3.5) can continue using ZooKeeper temporarily, but all new deployments should use KRaft mode exclusively.

Back to top

Final Thoughts

Kafka's architecture has undergone a major transformation. Where ZooKeeper once handled cluster coordination, KRaft now provides a self-contained, scalable, and more resilient control plane within Kafka itself.

If you are planning new Kafka deployments or modernizing existing clusters, you should migrate to KRaft mode as ZooKeeper-based clusters are now officially deprecated and unsupported as of Kafka 4.0.

For teams still running older environments, the time to plan your migration is now.

> ## *Deploy Kafka With Confidence*
>
> *OpenLogic provides SLA-backed, technical support for Apache Kafka, Kafka LTS, and full-service Kafka management— all delivered directly by experienced Enterprise Architects.*
>
> Explore Kafka Solutions ▶

Additional Resources

- Blog - IBM's Acquisition of Confluent: What It Means for Kafka's Future
- Blog - Kafka 4.1 Overview
- Webinar - Kafka Gone Wrong: How to Avoid Data Disasters
- Videos - How OpenLogic Supports Kafka
- Guide - Enterprise Kafka Resources

- Blog - Solving Complex Kafka Issues: Enterprise Case Studies
- Blog - Kafka Cluster Configuration Strategies
- Case Study - Credit Card Processing Company Avoids Kafka Exploit
- Blog - 8 Kafka Security Best Practices

Back to top

Solutions

ActiveMQ

AngularJS

Bootstrap

Databases

Enterprise Linux

Hadoop

Java

Kafka

Middleware

Spring

Tomcat

All Supported Technologies

Products

Hardened Linux Images

Long Term Support

Service Bundles

Technical Support

Customers

Services

Consulting Services

Implementations & Upgrades

Migration Services

Technical Account Management

Training

Resources

eBooks & Papers

Webinars

Videos

Blog

All Resources

© 2026 Perforce Software Inc. All Rights Reserved.

Privacy Policy | Terms of Use | Legal | Trust Center